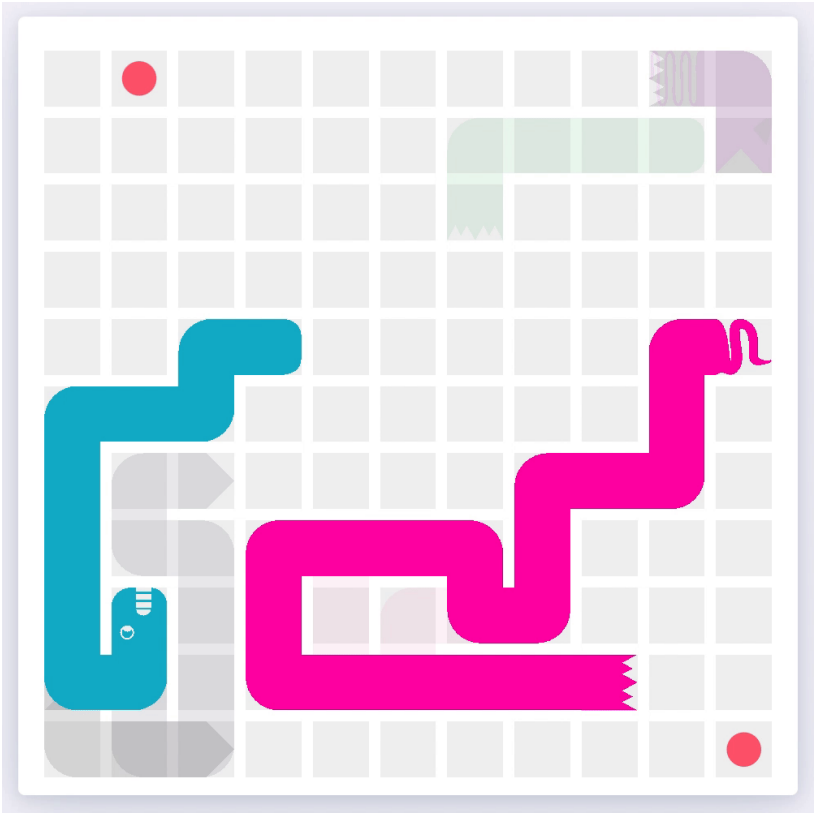
Lab 2 – Battlesnake Competition: Minimax Algorithm
100 points

Overview:
In this programming assignment, we will implement the Minimax adversarial search algorithm to control an AI to play a game of Battlesnake. This programming assignment should be completed individually. To complete the lab you must submit via Canvas a zip file containing your source code and a write-up describing your methods and results.



Part A: Tutorial

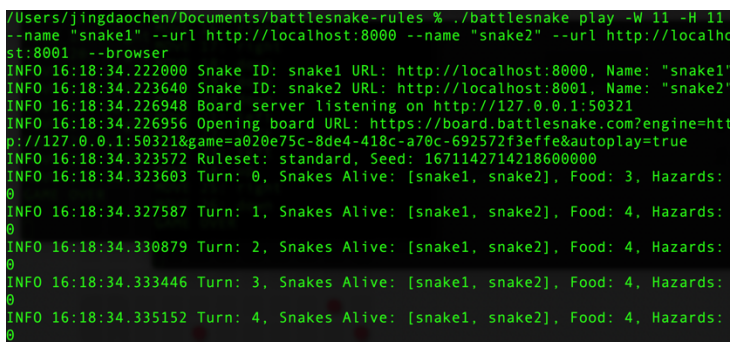
1. In the Battlesnake Competition, participants will build and deploy a Python AI script that implements the Battlesnake API (https://docs.battlesnake.com/api). When a game is created, the game engine will make HTTP requests to your Battlesnake server, sending game board information and asking for your next move. Your Battlesnake's behavior is determined by how you program it to act at each game state.
2. The game rules are as follows (refer https://docs.battlesnake.com/rules for the complete rules):
    - **Winning the Game:** Be the last snake alive on the game board
    - **Boundaries:** The game is played on a rectangular game board of variable size. Any snake attempting to move outside the boundaries will be eliminated from the game.
    - **Collisions:**
        - **Self-collisions:** If a snake collides with its own body, it will be eliminated

- **Body collisions:** If a snake collides with the body of another Battlesnake, it will be eliminated
- **Head-to-head collisions:** When two snakes meet head-to-head, the larger snake will survive and the shorter will be eliminated. If both are the same size, the snake with more health will survive. Otherwise, the outcome will be a draw (*this is a slightly modified version of the original implementation*).

- **Food:** At the beginning of each game some amount of food will be placed around the board. On each subsequent turn additional food may be added randomly. A snake that enters the same square as a piece of food will immediately consume it, filling its health to maximum and growing its length by one.
- **Health:** Each snake starts with a health of 100 points. At the end of every turn after making a move, a snake's health will decrease by 1 point. If a snake's health reaches 0, it will be eliminated. The only way to restore health is by consuming food.

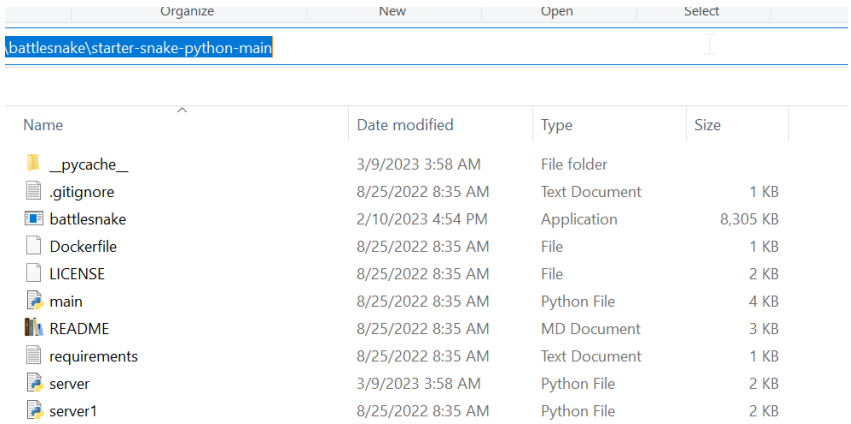
3. Read through the official guide (https://docs.battlesnake.com/guides/customizations) and quickstart (https://docs.battlesnake.com/quickstart) on Battlesnake's website to get familiarized with the game API, rules, and terminology. Note that only the **Standard** game mode will be relevant for this assignment.

Part B1: Playing on Localhost



In this section, we will run the game engine and code for competing Battlesnakes on the same computer on localhost. We will run two competing AI Battlesnakes in Python as well as the official Battlesnake rules engine.

1. Open the command line (terminal) in the target folder. In Windows, this can be easily done by typing *cmd* in the address bar https://stackoverflow.com/a/40146208 .
2. Go to https://github.com/BattlesnakeOfficial/rules/releases and select the compiled *battlesnake* executable that is suitable for your operating system (make sure to select the x86_64 option if your CPU uses the x86 architecture and arm64 option if your CPU uses the arm64 architecture). Download and extract the zip file to the target folder.
3. (optional) To compile the Battlesnake engine yourself, you will have to install the latest version of Go (https://go.dev/dl/). Then, download and compile the Go command line tool for the official Battlesnake rules and game logic from https://github.com/BattlesnakeOfficial/rules.
4. Download the files *main.py*, *simple.py* and *server.py* from the HW 2 page in Canvas.
5. Your project folder should look similar to the following:



6. Run the command *pip install flask* to install the Flask library as a dependency.
7. Open 2 additional terminal windows to have 3 terminal windows in total. Run the following commands to create 2 localhost servers with different port numbers. In the first terminal, run the *main.py* script which should be the main body of your minimax code. In the second terminal, run the *simple.py* script which provides a simple AI opponent.

```
python main.py --port 8000
python simple.py --port 8001
```

8. In the third terminal, run the following command (all in one line) to initiate a Battlesnake game. You may have to type `./battlesnake.exe` instead of `./battlesnake` when running on Windows. The game results will be animated through your web browser.

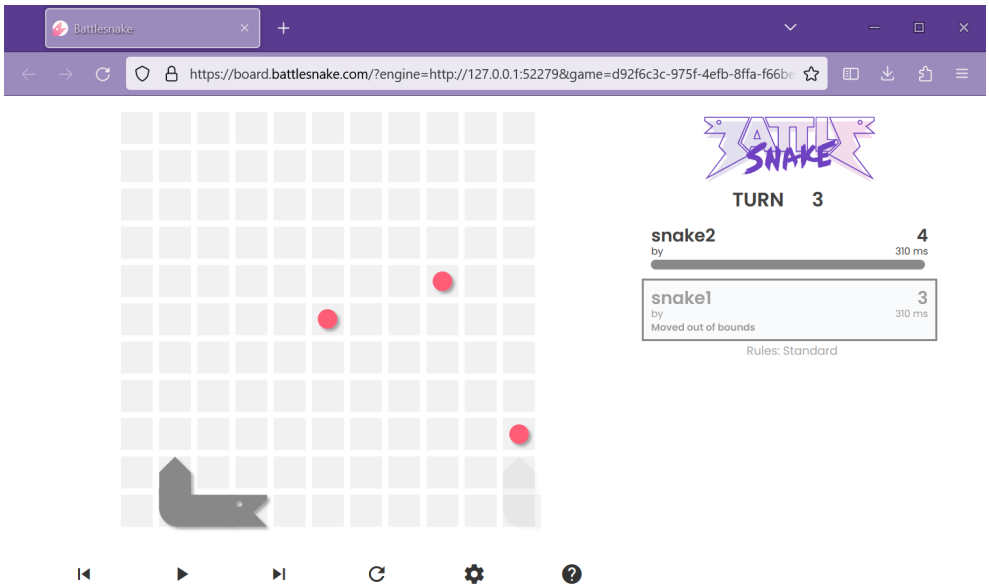
```
./battlesnake play -W 11 -H 11 --name "snake1" --url
http://127.0.0.1:8000 --name "snake2" --url http://127.0.0.1:8001
--browser
```

9. The starter Battlesnake only moves in a random direction. This means that for this first game, your Battlesnake is most likely going to collide with a wall or turn back in on its own body. To help your Battlesnake survive, you will need to implement the following functionality:
   i. Avoid colliding with walls
   ii. Avoid colliding with yourself
   iii. Avoid colliding with other snakes

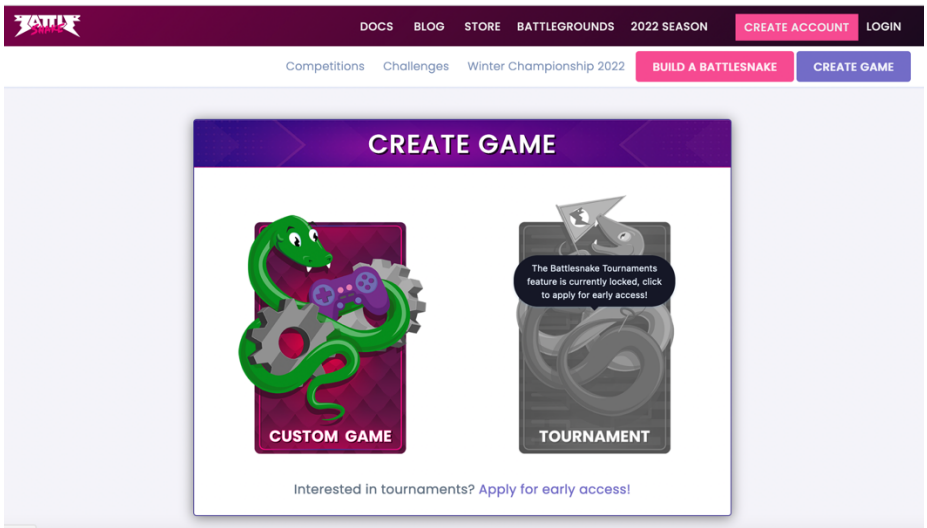
```
Microsoft Windows [Version 10.0.19045.2604]
(c) Microsoft Corporation. All rights reserved.

C:\Users\joe\battlesnake\starter-snake-python-main>python main.py --port 8000

Running Battlesnake at http://0.0.0.0:8000
 * Serving Flask app 'Battlesnake'
 * Debug mode: off

C:\Users\joe\battlesnake\starter-snake-python-main>python main.py

Running Battlesnake at http://0.0.0.0:8000
 * Serving Flask app 'Battlesnake'
 * Debug mode: off
INFO
GAME START
MOVE 0: right
MOVE 1: down
MOVE 2: right
GAME OVER
```

```
Microsoft Windows [Version 10.0.19045.2604]
(c) Microsoft Corporation. All rights reserved.

C:\Users\joe\battlesnake\starter-snake-python-main>python main.py --port 8001

Running Battlesnake at http://0.0.0.0:8001
 * Serving Flask app 'Battlesnake'
 * Debug mode: off
INFO
GAME START
MOVE 0: down
MOVE 1: right
MOVE 2: right
MOVE 3: up
GAME OVER
```

```
Microsoft Windows [Version 10.0.19045.2604]
(c) Microsoft Corporation. All rights reserved.

C:\Users\joe\battlesnake\starter-snake-python-main>battlesnake play -W 11 -H 11 --name "snake1" --url http://localhost:8000 --n
ame "snake2" --url http://localhost:8001 --browser
INFO 04:15:03.481360 Snake ID: df7a7fb7-7bbf-4560-a696-9f644a92ac71 URL: http://localhost:8000, Name: "snake1"
INFO 04:15:03.796886 Snake ID: 36b052da-2226-483f-9370-ac829917d274 URL: http://localhost:8001, Name: "snake2"
INFO 04:15:04.419859 Board server listening on http://127.0.0.1:52279
INFO 04:15:04.419859 Opening board URL: https://board.battlesnake.com?engine=http://127.0.0.1:52279&game=d92f6c3c-975f-4efb-8ff
a-f66be26e93e1&autoplay=true
INFO 04:15:04.611033 Ruleset: standard, Seed: 1678356903144054600
INFO 04:15:04.611116 Turn: 0, Snakes Alive: [snake1, snake2], Food: 3, Hazards: 0
INFO 04:15:04.921129 Turn: 1, Snakes Alive: [snake1, snake2], Food: 3, Hazards: 0
INFO 04:15:05.231868 Turn: 2, Snakes Alive: [snake1, snake2], Food: 3, Hazards: 0
INFO 04:15:05.543412 Turn: 3, Snakes Alive: [snake2], Food: 3, Hazards: 0
INFO 04:15:06.475858 Game completed after 4 turns. snake2 was the winner.

C:\Users\joe\battlesnake\starter-snake-python-main>
```



Part B2: Playing on the Official Server (optional)



Optionally, you may run the starter Python code using cloud hosting and play on the official server.

1. Download the Battlesnake Python Starter Project from Github. (https://github.com/BattlesnakeOfficial/starter-snake-python)
2. Identify a cloud provider that can host your Battlesnake code (https://codesandbox.io/ or https://replit.com).
    a. *Warning: most cloud providers require payment and some may block script access from the Battlesnake server*
    b. replit.com is the recommended option on the Battlesnake website but replit.com no longer offers free hosting as of 01/01/2024.
    c. Other cloud hosting options can be found here https://docs.battlesnake.com/guides/hosting-suggestions

3. Create an account on the cloud provider and upload your Python starter project there. After running your Battlesnake within the remote host, your Battlesnake server will start and you should see the live output from your Battlesnake server together with a public URL.
4. Create an account on https://play.battlesnake.com/. Go to "Build a Battlesnake" and paste in the Battlesnake server URL you have obtained from your cloud hosting site.
5. Go to "Create Game" and add your starter snake as one of the players. You may also add in one of the default AI snakes as your opponent. After clicking on "Start Game", you will be able to view the Battlesnake game on your web browser.

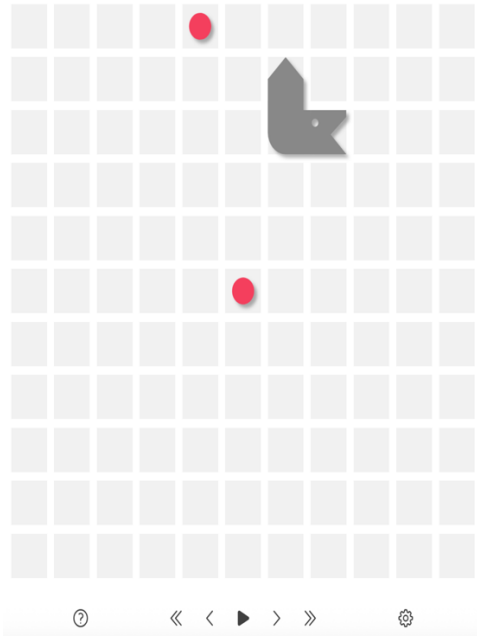
Part C: Game State (20 pts)

1. First, let's implement code to prevent your Battlesnake from collisions or going out of bounds. During each turn, your Battlesnake has a choice of 4 actions: {*up, down, left, right*}. Use the `game_state` variable to obtain the coordinates of your snake's head, your snake's body, the opponent's snake's body, and the boundaries of the game board. Flag any moves that would cause collisions as "unsafe" moves, then allow the snake to choose a move only among the "safe" moves.

```
is_move_safe = {"up": True, "down": True, "left": True, "right": True}

# We've included code to prevent your Battlesnake from moving backwards
my_head = game_state["you"]["body"][0]  # Coordinates of your head
my_neck = game_state["you"]["body"][1]  # Coordinates of your "neck"

if my_neck["x"] < my_head["x"]:  # Neck is left of head, don't move left
    is_move_safe["left"] = False

elif my_neck["x"] > my_head["x"]:  # Neck is right of head, don't move right
    is_move_safe["right"] = False

elif my_neck["y"] < my_head["y"]:  # Neck is below head, don't move down
    is_move_safe["down"] = False

elif my_neck["y"] > my_head["y"]:  # Neck is above head, don't move up
    is_move_safe["up"] = False

# TODO (Part C): Prevent your Battlesnake from moving out of bounds
# board_width = game_state['board']['width']
# board_height = game_state['board']['height']

# TODO (Part C): Prevent your Battlesnake from colliding with itself
# my_body = game_state['you']['body']

# TODO (Part C): Prevent your Battlesnake from colliding with the opponent's Battlesnake
# if len(game_state['board']['snakes']) == 2:
#     opponent_snake_index = 1 if game_state['board']['snakes'][0]['id'] == game_state['you']['id'] else 0
#     opponent_snake_body = game_state['board']['snakes'][opponent_snake_index]['body']

# Are there any safe moves left?
safe_moves = []
for move, isSafe in is_move_safe.items():
    if isSafe:
        safe_moves.append(move)

if len(safe_moves) == 0:
    print(f"MOVE {game_state['turn']}: No safe moves detected! Moving down")
    return {"move": "down"}
```

2. Documentation on what kinds of information is stored in the `game_state` variable is available on the Battlesnake API documentation (https://docs.battlesnake.com/api/example-move).

```
"board": {
  "height": 11,
  "width": 11,
  "food": [
    {"x": 5, "y": 5},
    {"x": 9, "y": 0},
    {"x": 2, "y": 6}
  ],
  "hazards": [
    {"x": 3, "y": 2}
  ],
  "snakes": [
    {
      "id": "snake-508e96ac-94ad-11ea-bb37",
      "name": "My Snake",
      "health": 54,
      "body": [
        {"x": 0, "y": 0},
        {"x": 1, "y": 0},
        {"x": 2, "y": 0}
      ],
      "latency": "111",
      "head": {"x": 0, "y": 0},
```

3. You may verify whether your collision checking code is working by running in single-player mode (note that in this case, checking collisions with the opponent's snake will not be available):
    a. Run the following two commands in two separate terminals:
    b. `python main.py --port 8000`
    c. `./battlesnake play -W 11 -H 11 --name "snake1" --url http://127.0.0.1:8000 --browser`

Part D: Evaluation Function (20 pts)

1. Next, let's implement an evaluation function that approximates the value of the current state. Remember that you can use the `game_state` variable to obtain the coordinates of your snake, your opponent's snake, and the location of food particles.
2. Uncomment the lines within the provided `evaluation_function` to implement a simple function that calculates the distance from the current snake position to the nearest food particle. Then in the main `move` function, loop through all move options, and pick the move that has the highest score based on the `evaluation_function`.
3. Run the Battlesnake game in single-player mode (as in Part C above), and you should be able to observe your snake chasing the food particles around the board.
4. Finally, customize your evaluation function so that it has the following attributes:
    a. The value should be a large positive number ($+\infty$) if the opponent's snake dies.
    b. The value should be a large negative number ($-\infty$) if your snake dies.
    c. The value should also be a function of various attributes of the game state such as the current number of move options available, length of each snake, number of squares controlled, snake health, distance to nearest food etc. Feel free to brainstorm and experiment with other factors that can be helpful to the survival of your snake.

Part E: Minimax (40 pts)

A key concept behind Adversarial AI is that instead of just optimizing our own agent's moves, we also need to account for the opponent's moves. To write a proper AI program that can win in a Battlesnake competition, we will implement the Minimax adversarial search algorithm. The Minimax algorithm will search for the best possible move given that your agent will attempt to maximize its score while the opponent will attempt to minimize your agent's score.

1. Implement the Minimax algorithm according to the pseudocode below. Use the `get_next_state` function to update the `game_state` variable whenever your snake makes a move or your opponent's snake makes a move. Use the `evaluation_function` to calculate the value of a particular game state when a leaf of the game tree is reached.
2. Note that the implementation of the `get_next_state` function does not update for collisions, add/remove food particles, calculate head-to-head collisions etc. by default, so you will have to implement that functionality yourself.
3. You may test the effectiveness of your Minimax Battlesnake by pitting it against the given simple AI opponent that makes random moves (simple.py) or any of the opponents on the official Battlesnake website. To run in two-player mode:
    a. Run the following three commands in three separate terminals:
    b. `python main.py --port 8000`
    c. `python simple.py --port 8001`
    d. `./battlesnake play -W 11 -H 11 --name "snake1" --url http://127.0.0.1:8000 --name "snake2" --url http://127.0.0.1:8001 --browser`
4. Make sure to customize your snake's name, color, head, and tail (https://docs.battlesnake.com/guides/customizations) so that you can visually distinguish your snake vs your opponent's snake.

```
function minimax(game_state, depth, is_maximizing_player):
    if depth = 0 or game_state is terminal:
        return evaluation_function(game_state)
    if is_maximizing_player then
        value := −∞
        best_move := None
        for each move in move_options do
            new_state = get_next_state(game_state, move, is_maximizing_player)
            value, best_move := max(value, minimax(new_state, depth-1, False))
        return (value, best_move)
    else # minimizing player
        value := +∞
        best_move := None
        for each move in move_options do
            new_state = get_next_state(game_state, move, is_maximizing_player)
            value, best_move := min(value, minimax(new_state, depth-1, True))
        return (value, best_move)
```

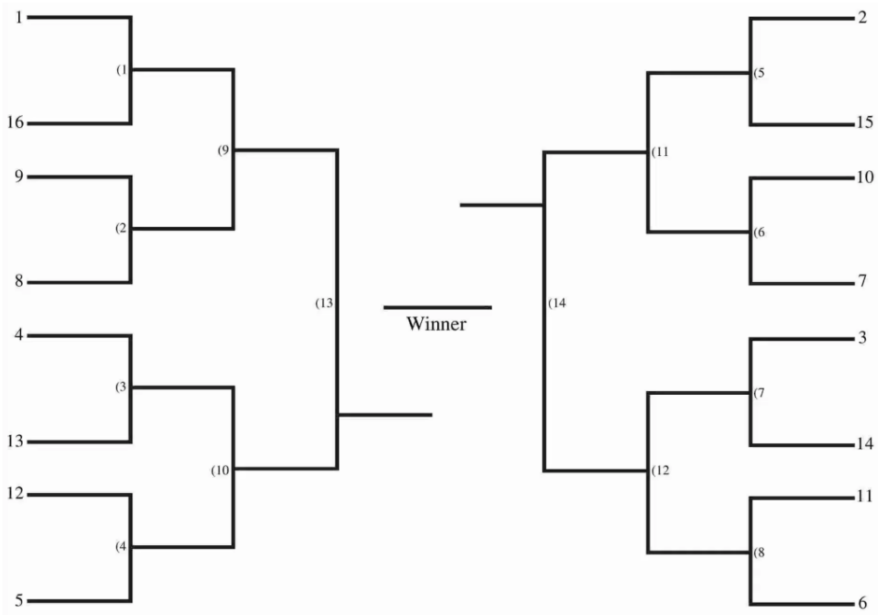
Part F: Discussion (20 pts)

Each student is required to submit a 1-page report in PDF format on Canvas with a brief discussion of the following items:

- Explanation of final algorithm implemented, with accompanying diagrams where applicable.
- Strategies that were attempted (both strategies that worked and did not work).
- List of external tools / libraries / resources that were used.
- Strengths and weaknesses of final algorithm implemented.
- Challenges encountered and how to overcome them.

Part G: Competition (optional)

1. The Battlesnake competition will be held in class (date will be specified on Canvas). The competition date will be right after the submission deadline for this assignment.
2. Entry in the Battlesnake competition is optional. If you would like to enter in the Battlesnake competition, submit your assignment as usual on Canvas but also leave a comment stating that you are participating. Because of the competition format, there is a limit of 16 total participants. If more than 16 entries are received, the earliest 16 entries submitted will be selected (based on the submission timestamp on Canvas).
3. A trial run of the Battlesnake competition will be conducted one week before the competition date. Participants will be randomly paired against one another. This will be an opportunity for participants to fix remaining bugs and strategize for the actual competition.
4. The final competition will be in a Single Elimination format with 1v1 battles. Each battle consists of multiple games with different initial seeds; the first snake to win 2 games advances to the next round (ignoring draws).
5. Participants that submit non-working code or code that violates the rules or fair play will be automatically disqualified. Their opponent will receive a bye to advance to the next round.
6. Winning participants will receive extra credit points according to the following bracket:
    a. Winner: +50 points
    b. Runner-up: +40 points
    c. Semi-finalist: +30 points
    d. Quarter-finalist: +20 points
    e. Consolation prize: +10 points



Tips and Tricks

1. When debugging your Battlesnake code against an AI opponent, it may be helpful to seed the pseudo-random number generator so that the results are reproducible (using the `--seed` command line argument). For example,

```
./battlesnake play -W 11 -H 11 --url http://127.0.0.1:8000 --seed 0
```

guarantees that the same board will be generated each time whereas

```
python simple.py --port 8000 –seed 0
```

guarantees that the simple AI opponent will execute the same moves each time given a certain board.

2. Go through the following sites to get more specific details about how the Battlesnake game logic is implemented:

- https://docs.battlesnake.com/guides/playing/rules
- https://github.com/BattlesnakeOfficial/rules/blob/main/standard.go

3. A key part of the Minimax algorithm lies in creating a new state dictionary that represents the "future" state of the game board after a specific snake has made a move, thereby updating the snake position and snake health in the state dictionary. The "future" state needs to be a "deep" copy of the "current" state so that each node in the minimax game tree essentially has its own copy of the game state. The provided skeleton code has a simple example `get_next_state()`, of how to implement this. However, creating a deep copy of the game state every iteration can be computationally expensive so you may experiment with ways to make this more efficient (e.g. modify only a single state at the time but backtrack / reverse those modifications when moving back up the game tree).

4. Even though the Minimax algorithm models the game as two players taking turns alternatingly, in reality, the Battlesnake game engine executes moves for both snakes simultaneously. This means that your code will have to carefully consider edge cases such as when a snake moves to an opponent's tail or to its own tail or when both snakes move to the same location. Also note that your snake will not always be at index 0 in the game state, instead, your snake ID is given by `state["you"]["id"]`.

5. Consider using some of the AI techniques we have learned in this class such as state-space search, local search, heuristics, alpha-beta pruning, Monte-Carlo tree search etc. to optimize the performance of your Battlesnake. Make sure to use efficient data structures and optimized algorithms to minimize computation time.

6. Other helpful resources:

- Official Battlesnake tools and documentation
  - https://github.com/BattlesnakeOfficial/
- A curated list of open-source Battlesnake projects
  - https://github.com/xtagon/awesome-battlesnake
- Paper on Battlesnake algorithms
  - https://dl.gi.de/bitstream/handle/20.500.12116/29001/SKILL2019-09.pdf

Frequently-asked Questions

Q: Can I refer to or make use of example code from the Internet?
A: Yes, you may use any Python library functions, Github, StackOverflow, or open-source community-developed Battlesnake projects that are available on the Internet (e.g. functions for geometry calculations, game update logic, path planning). However, this may only be in the form of function imports from your main Python script and all the code in *main.py* (i.e. the implementation of the Minimax algorithm) has to be your own work. You must cite and acknowledge all external resources that you use in your written PDF report that you submit on Canvas by explaining how you used those resources and providing links to the original sites.

Q: What programming languages / frameworks are allowed?
A: To make the execution time fair for all teams, the final Battlesnake code submitted should be written in Python, although your code for simulation and benchmarking can be written in other languages. The code should be written so that it can be executed on a basic laptop computer. During the competition, submitted code from all teams will be executed on the instructor's laptop. Standard Python numerical computing libraries such as NumPy and SciPy are allowed but Python libraries that exploit hardware/GPU acceleration/parallel computing such as Tensorflow and PyTorch are not allowed. Please check with the instructor if you are unsure whether a specific library is allowed.

Q: What are the ruleset and game settings that will be used for the Battlesnake competition?
A: The Battlesnake competition will use the Standard ruleset with a 11 x 11 board and a 500ms timeout.